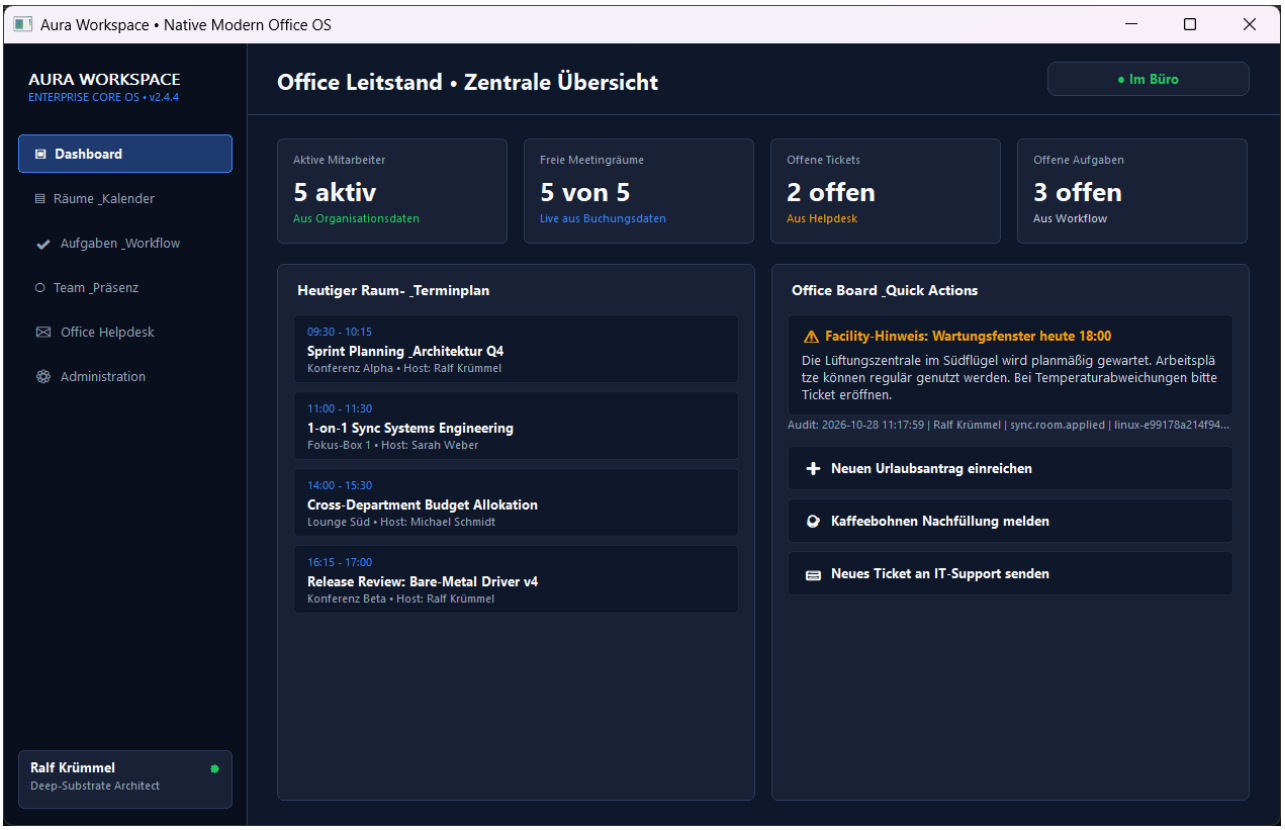
AURA WORKSPACE ENTERPRISE

Native Unternehmenssoftware über Windows, Linux und Android

Technisches Whitepaper | Release v2.4.5 · Live-Teststand v2.4.4 | 28. Oktober 2026

Entwickler: Ralf Krümmel • KR Compiler IDE / KR Unified Language



Titelabbildung: Windows-Dashboard nach erfolgreicher Übernahme des unter Ubuntu angelegten fünften Raums.

Executive Summary

AURA Workspace Enterprise ist eine implementierte plattformübergreifende Unternehmensplattform für Räume und Ressourcen, Projekte und Aufgaben, Mitarbeiter und Organisation, Helpdesk sowie zentrale Mehrbenutzer-Synchronisation. Die Architektur kombiniert C++-Datenbank- und Anwendungskern, Rust-Geschäftsregeln, Go-Dienste und SQLite. Die Workspace-Anwendungen stehen für Windows und Linux zur Verfügung, eine Android-App ist vorhanden. Der zentrale AURA-Server existiert sowohl für Windows als auch für Linux. Dieses Whitepaper dokumentiert insbesondere den erfolgreichen Live-Nachweis einer Raumdatensynchronisation von Ubuntu nach Windows. Am 28. Oktober 2026 wurde zusätzlich Release v2.4.5 auf GitHub veröffentlicht: mit unter Ubuntu erstellten Paketen für Linux, Windows und Android sowie einem SHA-256-Manifest. Der dokumentierte v2.4.5-Build verwendet die KR Compiler IDE v4.3.1 als eigenentwickelte, offlinefähige Multi-Target-Werkzeugkette. Der Ablauf ist im Projekt als reproduzierbarer Entwicklerleitfaden veröffentlicht.

Zwei voneinander getrennte Live-Testketten sind dokumentiert: Am 27. Oktober 2026 erstellte eine Android-App über einen USB-Tunnel und HTTPS erfolgreich einen Raum am Windows-Zentralserver, der anschließend im Windows-Workspace angezeigt wurde. Am 28. Oktober 2026 wurde ein auf Ubuntu lokal erstellter Raum über den Ubuntu-Zentralserver an einen getrennten Windows-Client übertragen und dort sichtbar übernommen.

1 Zielsetzung und Einsatzmodell

AURA stellt eine gemeinsame Arbeitsumgebung für verschiedene Unternehmensrollen und Geräte bereit: mobile Mitarbeiter mit Android-App, Büroarbeitsplätze unter Ubuntu und Managementarbeitsplätze unter Windows. Der zentrale Server ist für Linux und Windows implementiert und verwaltet gemeinsame Fachobjekte. Lokale Datenbanken ermöglichen ausgewählte Offline-Abläufe. Authentifizierung und Berechtigungen trennen lokale Benutzerkonten und zentrale Serveridentitäten.

Die Anforderung ist ausdrücklich funktional und nicht nur kompilierungstechnisch: Eine Änderung muss auf Gerät A erfasst, zum Server übertragen und auf Gerät B ohne erneute Eingabe sichtbar werden. Dieser Anspruch wurde für Raumstammdaten praktisch bestätigt; für alle anderen Fachbereiche ist der gleiche Nachweis nicht Gegenstand dieses Whitepapers.

2 Architektur und Technologie

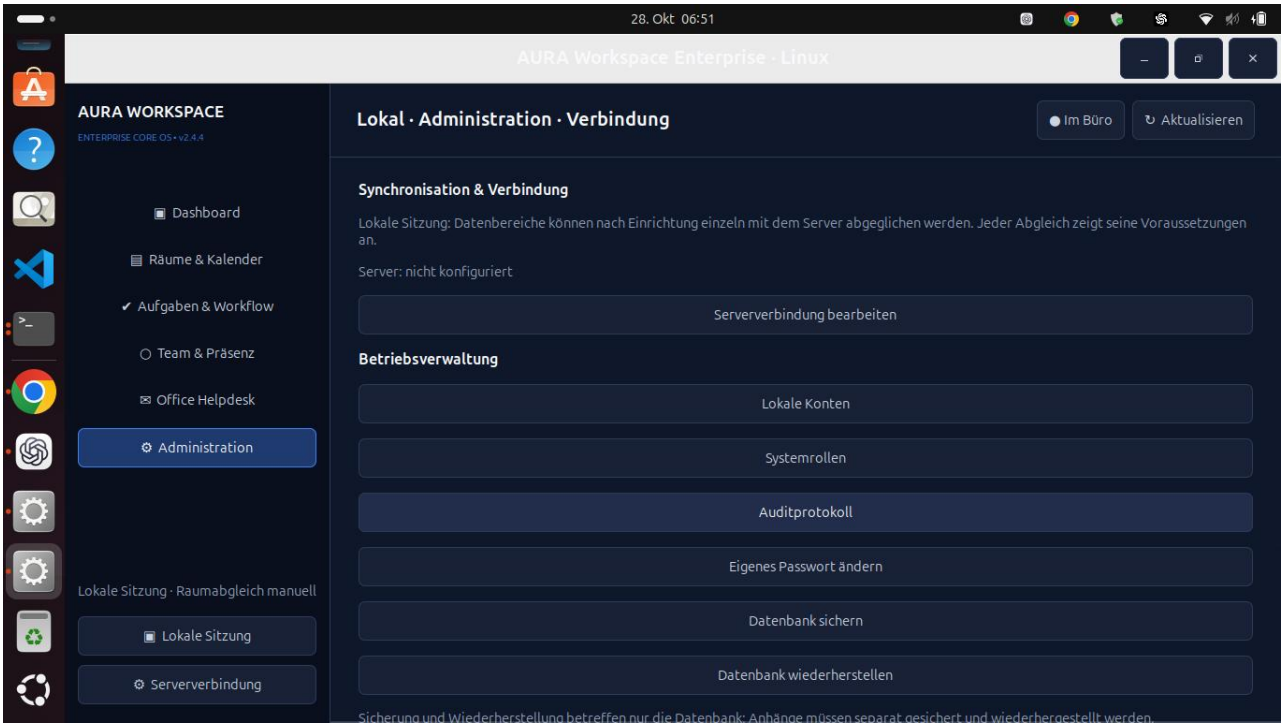
Windows-Client	**Native Win32-/GDI-Oberfläche, C++-Anwendungslogik, lokale SQLite-Datenbank**
Linux-Client	Native GTK-basierte Oberfläche, portierter gemeinsamer C++-Datenbankkern, lokale SQLite-Datenbank
Zentralserver	Zentraler Go-Server für Windows und Linux; REST-API, SQLite-Persistenz und Fachereignisse
Validierung	Rust-Regeln über definierte C-ABI
Synchronisation	Persistente Outbox, Serverereignisse, Cursor-Feed, externe Objektkennungen
Buildumgebung	KR Compiler IDE / KR Toolchain; plattformübergreifende Artefakterstellung

Ein zentrales Element ist die Trennung lokaler Primärschlüssel von externen Objektkennungen. So können Clients unabhängig erzeugte IDs besitzen und dennoch denselben serverseitigen Datensatz zuordnen. Das Windows-Projekt verwendet dafür eine Zuordnungstabelle für synchronisierte Fachobjekte.

3 Fachlicher Funktionsumfang

Die Projektdokumentation nennt Raumstammdaten, Suche, Ausstattung, Kapazität, Einzel- und Serienbuchung, Umbuchung, Stornierung und Sperrzeiten. Dazu kommen Projekt- und Aufgabenverwaltung mit Fälligkeiten, Abhängigkeiten, Unteraufgaben und Freigaben, Mitarbeiter- und Organisationsverwaltung, Präsenz, Urlaubsanträge, Helpdesk-Tickets, Facility-Assets und Wartungsaufträge sowie Konten, Rollen, Audit und Backups.

Der übergreifende Produktaufbau ist vorhanden: Windows-Workspace, Linux-Workspace, Android-App und zentrale Serverimplementierungen für Windows und Linux. Die folgenden Nachweise unterscheiden zwischen dokumentiertem Produktumfang, automatisierten Prüfungen und den in diesem Whitepaper konkret beobachteten Live-Vorgängen. Nicht einzeln vorgeführte Funktionen werden dadurch nicht als fehlend bezeichnet.



Linux-Client: Administrations- und Verbindungsansicht im lokalen Modus vor der Konfiguration.

4 Entwicklungsphasen und Aufwand

Phase 1 · Enterprise-Release v2.4.1	**7 Stunden 26 Minuten**
Phase 2 · Ausbau Linux v2.4.4	11 Stunden 30 Minuten
Summe beider dokumentierten Phasen	18 Stunden 56 Minuten

Die erste Entwicklungsphase brachte den veröffentlichten Enterprise-Release v2.4.1 mit Windows-Komplettpaket, Linux-ELF, zwei signierten Android-APKs und SHA-256-Manifest hervor. Die zweite Entwicklungsphase erweiterte insbesondere den Linux-Workspace um den portierten Datenbankkern, GUI-Arbeitsabläufe, zentrale Buchungen, Synchronisation und Administration. Der AURA-Zentralserver steht sowohl unter Windows als auch Linux zur Verfügung. Der ausgewiesene Zeitaufwand bezieht sich auf diese beiden Phasen, nicht auf die historische Entstehung der KR-Toolchain oder vorhandene Komponenten. Der anschließende Release v2.4.5 entstand mit einer vereinheitlichten Ubuntu-Buildpipeline für alle drei Zielplattformen; die dokumentierten 18 Stunden und 56 Minuten beziehen sich weiterhin ausschließlich auf die beiden zuvor erfassten Entwicklungsphasen.

5 Prüfstrategie: Von Regression zu Echtbetrieb

Vor dem Gerätevergleich wurden in Entwicklungsprotokollen erfolgreiche Go-, Datenbank- und Berechtigungstests gemeldet. Unter Ubuntu wurde der native Server mit /health (HTTP 200) verifiziert. Die serverseitige Identitätsabfrage antwortete mit gültigem Bearer-Token ebenfalls mit HTTP 200. Der Windows-Rechner verband sich über einen verschlüsselten SSH-Tunnel zum ausschließlich auf Ubuntu-Loopback gebundenen Server.

Der Tunnel leitete Windows-Loopback-Port 18787 auf Ubuntu-Loopback-Port 8787 weiter. Dadurch wurde der lokale HTTP-Server nicht direkt im LAN exponiert. Dieser konkrete Versuch ist kein Nachweis für direktes HTTPS zwischen Windows und Ubuntu; ein gesonderter Android-HTTPS-Test über USB-Tunneling war zuvor berichtet worden.

5.1 Live-Nachweis: Android → Windows-Zentralserver → Windows

Am 27. Oktober 2026 wurde der Windows-Zentralserver lokal über HTTPS unter https://localhost:8787 betrieben. Die Android-App wurde von einem realen Smartphone über den USB-Tunnel adb reverse tcp:8787 tcp:8787 mit der Zentrale verbunden. Ein Synchronisationsaufruf lieferte HTTP 200; die Raumerstellung auf dem Smartphone wurde mit HTTP 201 bestätigt.

Testobjekt: „HTTPS-Live-Test-Raum“, Standort Berlin-HQ, Kapazität 12 Personen. Anschließend wurde der Raum im sichtbaren Windows-Workspace über die HTTPS-Zentrale synchronisiert und mit seinen Stammdaten angezeigt. Zusätzlich wurden die signierte Android-APK und der gesonderte HTTPS-Teststand im Entwicklungsprotokoll genannt (Commit 1f36244).

Dieser Versuch belegt einen realen mobilen Datenweg über einen Windows-Server. Die verwendete Test-CA und das Zertifikat waren auf die lokale USB-Testumgebung beschränkt; daraus wird keine Freigabe für öffentlich erreichbaren HTTPS-Produktivbetrieb abgeleitet.

6 Reproduzierbarer Live-Test: Ubuntu → Zentralserver → Windows

6.1 Ausgangslage

Ubuntu und Windows liefen auf unterschiedlichen physischen Rechnern. Die Ubuntu-GUI war als lokale Administratorsitzung gestartet. Die Windows-Anwendung verwendete ihre lokale SQLite-Datenbank und ein zuvor eingerichtetes lokales Konto; die zentrale Authentifizierung erfolgte separat mittels Bearer-Token.

6.2 Testobjekt und Erfassung

Raumname	**UBUNTU-WINDOWS-SYNC-TEST-01**
Standort	Ubuntu-Teststandort
Kapazität	8 Personen
Ausstattung	Display, Whiteboard
Quellsystem	Lokaler Ubuntu-Workspace

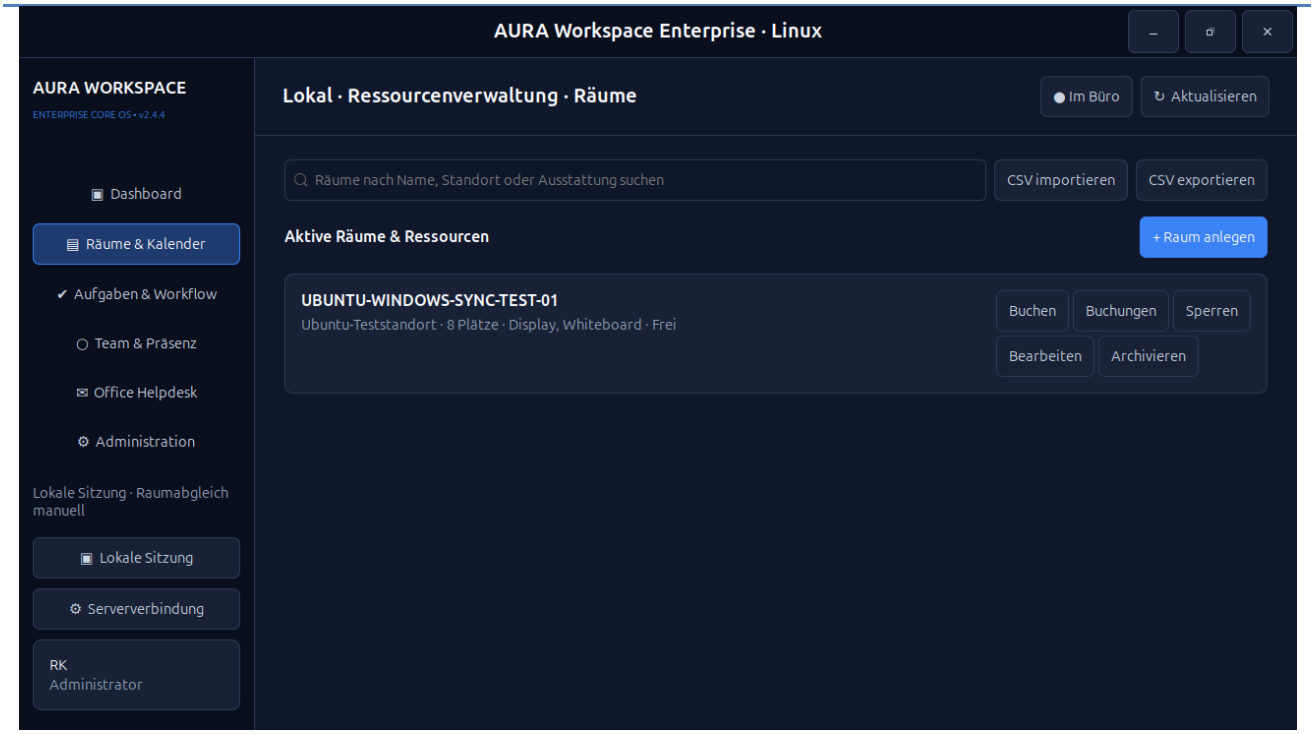


Abbildung 1: In der lokalen Ubuntu-Raumverwaltung angelegtes Testobjekt.

6.3 Übertragung und Serverabgleich

Unter Administration → Fachbereiche abgleichen wurde die Raumübertragung ausgelöst. Nach Bestätigung der Bindung an das Serverkonto meldete AURA: „1 Änderungen übertragen, 4 Räume übernommen. Andere lokale Daten wurden nicht synchronisiert.“ Damit war die erfolgreiche Raumübertragung und gleichzeitige Übernahme vorhandener zentraler Räume in Ubuntu dokumentiert.



Abbildung 2: Ubuntu bestätigt den abgeschlossenen Raumabgleich mit einer gesendeten Änderung und vier übernommenen Räumen.

6.4 Übernahme unter Windows

Nach dem Start des Windows-Clients mit AURA_SYNC_URL und dem gültigen zentralen Token erschien der zuvor auf Ubuntu angelegte Raum in der Windows-Raumverwaltung als zusätzlicher fünfter Raum. Name, Standort, Kapazität und Ausstattung stimmten mit dem Quellsystem überein.

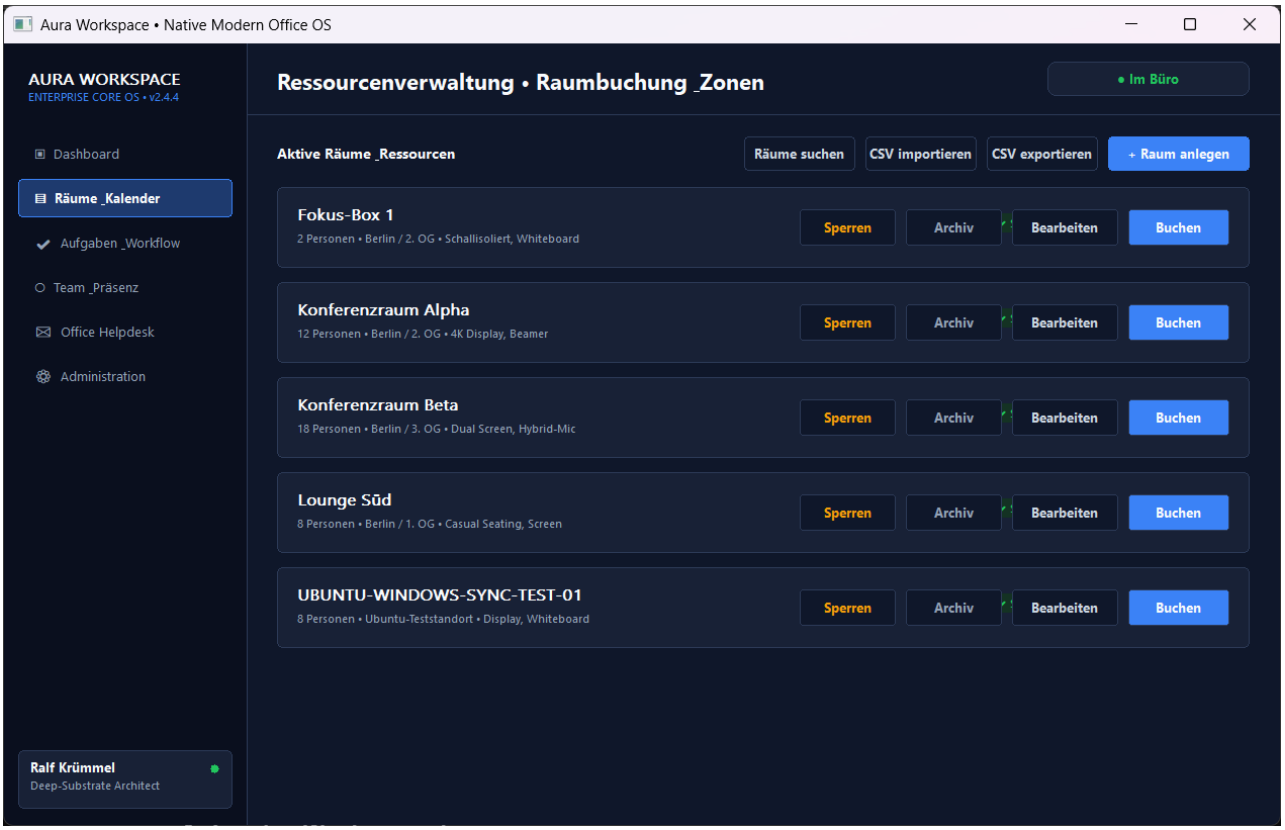


Abbildung 3: Windows-Raumverwaltung mit vier Bestandsräumen und dem übernommenen Ubuntu-Testobjekt.

Das Windows-Dashboard zeigte anschließend „5 von 5“ verfügbare Meetingräume. Im Audit erschien ein Ereignis der Form „sync.room.applied“. Damit ist sowohl die lokale Datenübernahme als auch ihre Berücksichtigung durch die native GUI erkennbar. Ein zuvor gezeigter Screenshot mit „4 von 4“ stammte aus einer älteren Windows-Zwischenablage und stellt ausdrücklich keinen Synchronisationsdefekt dar.

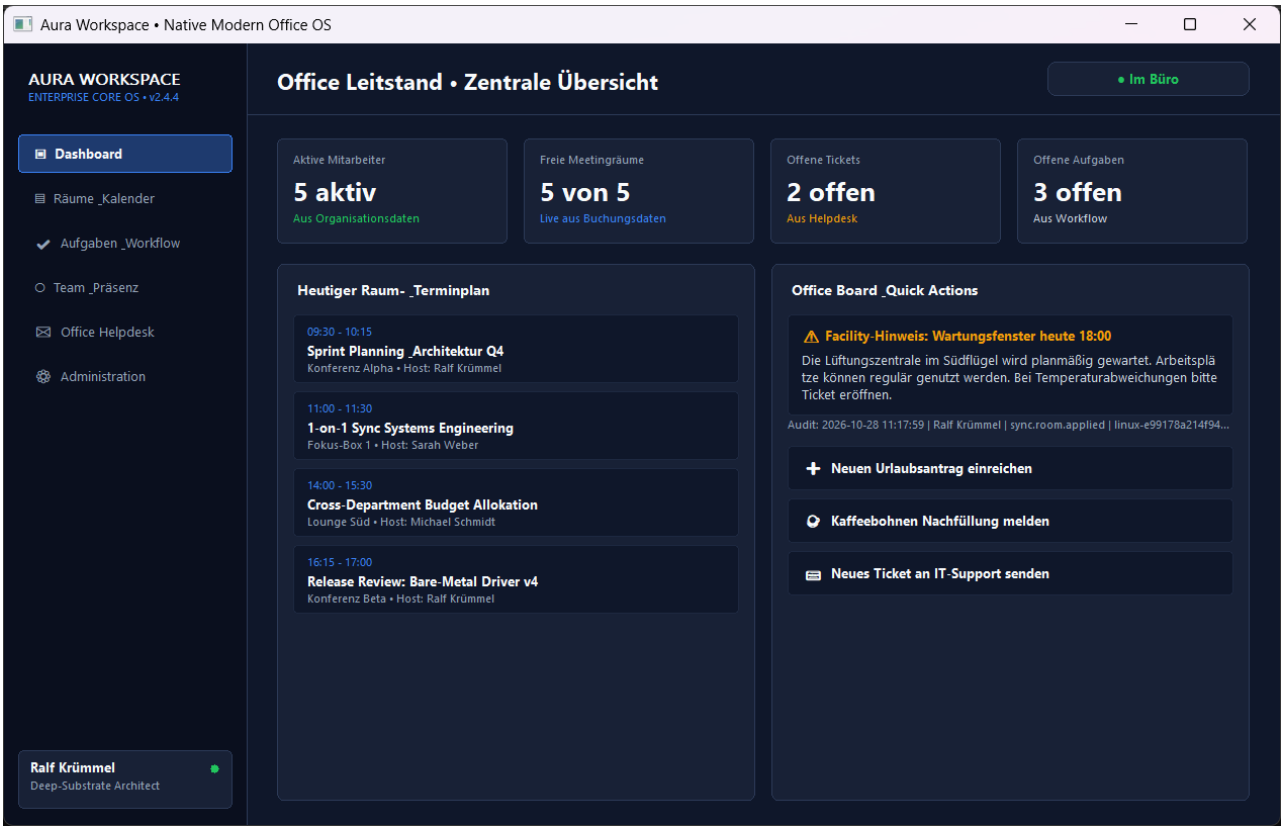


Abbildung 4: Aktualisiertes Windows-Dashboard mit fünf verfügbaren Räumen und Audit-Hinweis auf den übernommenen Datensatz.

7 Resultat und Aussagekraft

Die realen Tests belegen zwei konkrete plattformübergreifende Datenwege: Android-App → Windows-Zentralserver → Windows-Workspace (27.10.2026) sowie Ubuntu-Workspace → Ubuntu-Zentralserver → Windows-Workspace (28.10.2026). Im zweiten Test blieb der vorhandene Windows-Bestand erhalten; der neu eingegangene Raum wurde sichtbar ergänzt und mit sync.room.applied auditiert.

Der Live-Test belegt den hier dargestellten Raumdatenprozess auf zwei physischen Rechnern. Er ist kein einzelner Vollständigkeitstest aller bereits implementierten Module oder der Android-App. Diese Einschränkung betrifft ausschließlich die Reichweite des dokumentierten Tests, nicht die Existenz der jeweiligen Anwendungen oder der Windows- und Linux-Serverimplementierung.

8 Sicherheits- und Betriebsaspekte

Die Plattform dokumentiert tokengebundene Identitäten, Rollen-/Fachberechtigungen, protokollierte Änderungen und Integritätsprüfungen. Der Live-Test brachte außerdem eine praktische Trennung ans Licht: Ein lokales Windows-Konto muss initial eingerichtet sein, unabhängig davon, ob der zentrale Token gültig ist. Ebenso müssen Token und URL der beiden Betriebssysteme exakt zum verwendeten Server passen.

Der gewählte SSH-Tunnel eignet sich als kontrollierter lokaler Nachweis. Für regulären Unternehmensbetrieb sind eine dauerhaft administrierte Netzwerkanbindung, verifiziertes TLS, sichere Tokenbereitstellung und eine geregelte Sicherungs-/Wiederherstellungsstrategie nötig. Ein Server-Token darf nicht in Quellcode, Screenshots oder öffentlichen Repositories abgelegt werden.

9 Release v2.4.5: Ubuntu als plattformübergreifende Buildplattform

Am 28. Oktober 2026 wurde AURA Workspace Enterprise v2.4.5 im GitHub-Repository veröffentlicht. Anders als beim ursprünglichen Windows-zentrierten Build ermöglichen neu ergänzte Shell-Skripte die Erstellung und Paketierung von Linux-, Windows- und Android-Artefakten unter Ubuntu. Das Skript build-enterprise.sh führt die drei Plattform-Builds zusammen und aktualisiert das Release-Manifest.

Verifizierter GitHub-Stand: Tag v2.4.5; Commit auf main 1eb745149466421c33da7dbc58844779ce8765f6 („Release version 2.4.5: Multi-platform builds for Linux, Windows and Android“); Veröffentlichungszeitpunkt 28.10.2026, 12:22:34 UTC. GitHub-Adresse: https://github.com/kruemmel-python/AURA-Workspace-Enterprise/releases/tag/v2.4.5

Veröffentlichte Release-Artefakte

• Linux-Paket: AURA-Workspace-Enterprise-v2.4.5-linux-x64.tar.gz (12.508.220 Bytes) – Workspace und Server-Paket.

• Linux-ELF: aura-workspace-enterprise (10.196.564 Bytes).

• Windows-Paket: AURA-Workspace-Enterprise-2.4.5-windows-x64.zip (16.119.073 Bytes) – Workspace, Laufzeitkomponenten und Synchronisationsserver.

• Windows-EXE: AURA-Workspace-Enterprise.exe (3.094.528 Bytes).

• Android-App: aura-workspace-mobile.apk (29.080 Bytes), signiertes Release-Artefakt laut Buildprotokoll.

• Integritätsdatei: release-manifest.json (573 Bytes).

Nachweisabgrenzung: Die Veröffentlichung der sechs GitHub-Assets sowie Tag und Commit wurden überprüft. Der Buildabschluss und die Signaturprüfung beruhen auf dem bereitgestellten Entwicklungsprotokoll; ein erneuter Installations- oder Laufzeittest speziell der v2.4.5-Artefakte wurde im Rahmen dieses Whitepapers nicht durchgeführt. Die besonders kleine Android-APK sollte im Release-Test auf einem Gerät installiert sowie auf Update-Signaturkompatibilität geprüft werden.

9.1 KR Compiler IDE v4.3.1 als Build- und Integrationsbasis

Der am 28. Oktober 2026 ergänzte Build- und Integrationsleitfaden beschreibt den konkret verwendeten Weg, AURA vollständig unter Ubuntu zu bauen. Die im Verzeichnis toolchain/ bereitgestellten und versionierten Werkzeuge umfassen Clang/LLVM mit LLD, Go, Rust, Windows-Zielbibliotheken, Android NDK/SDK und JDK. Das KR-Projektmanifest aura.krproject definiert die gemeinsamen Modulquellen und Zielplattformen; KrPipelineVerifier.java dient der vorgeschalteten Prüfung der KR-Projektpipeline.

Die fachlichen Komponenten sind über feste C-ABI-Grenzen miteinander verbunden: C++20 implementiert den gemeinsamen SQLite-Datenbankkern und Windows-GUI-/Linux-Bridge-Funktionen; Rust implementiert die Regelprüfung; Go betreibt den Synchronisationsserver und erstellt unter Windows auch die Service-DLL; Java und Android-Buildwerkzeuge erzeugen die mobile Touch-App.

9.2 Ein Gesamtbuild auf Ubuntu

Der dokumentierte Buildaufruf lautet im AURA-Projektverzeichnis: ./build-enterprise.sh. Er orchestriert die Linux-, Windows- und Android-Builds sowie die Erstellung des SHA-256-Release-Manifests. Die plattformspezifischen Skripte liegen unter clients/linux/, clients/windows/ und clients/android/.

Linux x64: CMake, C++/SQLite und Rust sowie Go erzeugen Workspace, native Bridge und Zentralserver; paketiert als .tar.gz. Windows x64: Clang/LLD und Windows-Sysroot, Rust für x86_64-pc-windows-gnu und Go-Cross-Build erzeugen Windows-EXE, DLLs und den Windows-Zentralserver; paketiert als ZIP. Android: aapt2, javac, d8 und apksigner erzeugen die signierte Touch-APK. Die Arbeitsumgebung verwendet dafür die vorab bereitgestellten Toolchains ohne Netzwerkdownloads während des Builds.

9.3 Offizielle Veröffentlichung von Compiler und Produkt

AURA Workspace Enterprise v2.4.5 — https://github.com/kruemmel-python/AURA-Workspace-Enterprise/releases/tag/v2.4.5

KR Compiler IDE v4.3.1 — https://github.com/kruemmel-python/KR-Compiler-Windows-x64-v4.3.1/releases/tag/v4.3.1

Der KR-Compiler-Release v4.3.1 umfasst laut GitHub Windows-x64-ZIP, Linux-x64-Teilar-chive samt Zusammenbauanleitung, Android-IDE-APK und SHA256SUMS. Der AURA-Release v2.4.5 veröffentlicht sechs Assets: Linux-Bundle und ELF, Windows-Bundle und EXE, Android-Mobile-APK sowie Release-Manifest. Damit sind Compiler-Werkzeugkette und damit erstelltes Unternehmensprodukt separat versioniert und veröffentlicht.

9.4 Dokumentation im Quellrepository

Der projektinterne Leitfaden docs/kr-compiler-build-guide.md dokumentiert Architektur, verwendete Toolchain, konkrete Kompilier- und Linkerschritte, Paketierungsbefehle und das integrierte Manifest. Er wurde vom ausführenden Entwicklungsagenten anhand der durchgeführten Arbeitsschritte erstellt und im README des AURA-Repositorys verlinkt.

10 Produktstatus und Nachweisabgrenzung

AURA Workspace Enterprise liegt nach dem dokumentierten Entwicklungsstand als plattformübergreifendes Gesamtsystem vor: native Workspace-Anwendungen für Windows und Linux, eine Android-App sowie Zentralserver für beide Desktop-Plattformen. Die beiden realen Live-Tests belegen die konkret beschriebenen Android–Windows- und Ubuntu–Windows-Abläufe. Diese Nachweisabgrenzung bedeutet nicht, dass andere bereits implementierte Produktfunktionen fehlen. Die Version v2.4.5 ist zusätzlich als GitHub-Release mit sechs verfügbaren Assets verifiziert; die Existenz eines Assets ersetzt keinen gesonderten Installations- und Laufzeittest.

11 Fazit

AURA Workspace Enterprise demonstriert eine implementierte native Unternehmensplattform auf Grundlage der KR Compiler IDE mit Windows- und Linux-Servern, Windows- und Linux-Workspaces sowie Android-App. Der HTTPS-Livetest vom 27. Oktober belegt eine mobile Raumerstellung über den Windows-Zentralserver bis zur sichtbaren Windows-Oberfläche. Der Live-Test vom 28. Oktober bestätigt zusätzlich die lokale Erfassung unter Ubuntu, den zentralen Raumabgleich und die korrekte Übernahme in eine native Windows-Datenbank und -Oberfläche. Zusammen liefern diese beiden Tests nachvollziehbare, reale Plattformnachweise. Mit der Veröffentlichung von v2.4.5 ist darüber hinaus eine plattformübergreifende Release-Pipeline unter Ubuntu dokumentiert und das Ergebnis auf GitHub verfügbar. Die unter Ubuntu ausgeführte KR-Buildpipeline ist zusätzlich ein konkreter Praxiseinsatz des eigenentwickelten Compilers für Windows, Linux und Android. Beide Projekte besitzen eigene offizielle GitHub-Releases.

Dokumentationsbasis

Benutzerseitige Abschluss- und Entwicklungsprotokolle vom 26.–28. Oktober 2026; AURA Workspace Enterprise README; bereitgestellte Code-Dumps; Bildschirmaufnahmen aus Ubuntu und Windows; Terminalausgaben der Linux-Server- und Windows-Tunnel-Tests. Der Text unterscheidet zwischen berichteten automatisierten Tests und beobachteten Live-Ergebnissen. Keine unabhängige Sicherheitszertifizierung oder externe Produktabnahme wird behauptet. Ergänzend: GitHub-Release v2.4.5, Tag und Commit auf dem Repository kruemmel-python/AURA-Workspace-Enterprise, geprüft am 28.10.2026. Ergänzend: docs/kr-compiler-build-guide.md (KR Compiler IDE v4.3.1) sowie die offiziellen GitHub-Releases von AURA v2.4.5 und KR Compiler IDE v4.3.1.